

HW7 report

楊文德

程式介紹

1. BFS function

```
int BFS(vector<vector<int>> &G)
{
    int num = 0;
    vector<vector<int>> Visited(8, vector<int>(8, false));
    queue<pair<int, int>> q;

    q.push({0, 0});
    Visited[0][0] = true;

    while (!q.empty())
    {
        pair<int, int> tmp = q.front();
        q.pop();

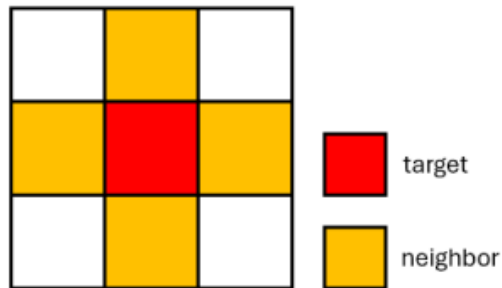
        if (tmp.first - 1 >= 0 && Visited[tmp.first - 1][tmp.second] == false)
        {
            q.push({tmp.first - 1, tmp.second});
            Visited[tmp.first - 1][tmp.second] = true;
        }
        if (tmp.first + 1 < 8 && Visited[tmp.first + 1][tmp.second] == false)
        {
            q.push({tmp.first + 1, tmp.second});
            Visited[tmp.first + 1][tmp.second] = true;
        }
        if (tmp.second - 1 >= 0 && Visited[tmp.first][tmp.second - 1] == false)
        {
            q.push({tmp.first, tmp.second - 1});
            Visited[tmp.first][tmp.second - 1] = true;
        }
        if (tmp.second + 1 < 8 && Visited[tmp.first][tmp.second + 1] == false)
        {
            q.push({tmp.first, tmp.second + 1});
            Visited[tmp.first][tmp.second + 1] = true;
        }

        if (G[tmp.first][tmp.second] == 1)
        {
            num++;
            EraseLand(G, tmp.first, tmp.second);
        }
    }

    return num;
}
```

在 BFS 函數中我使用到 BFS 的架構，透過 queue 流程化節點的搜尋，將 BFS 的實踐分成三個步驟：

1. 先用 front 取得 queue 前面的元素再 pop 掉。
2. 再依據 neighbor 的定義掃過周圍相鄰的位置，如果未被搜索過就加入倒 queue 中。
3. 如果當前的節點是 1(陸地)，先將大陸的數量+1，再呼叫 EraseLand 將相鄰的陸地通通改成 0(水域)，以防重複計算到陸地。



2. EraseLand function

```
void EraseLand(vector<vector<int>> &G, int x, int y)
{
    queue<pair<int, int>> q;
    q.push({x, y});

    while (!q.empty())
    {
        pair<int, int> tmp = q.front();
        q.pop();

        if (tmp.first - 1 >= 0 && G[tmp.first - 1][tmp.second] == 1)
        {
            q.push({tmp.first - 1, tmp.second});
            G[tmp.first - 1][tmp.second] = 0;
        }
        if (tmp.first + 1 < 8 && G[tmp.first + 1][tmp.second] == 1)
        {
            q.push({tmp.first + 1, tmp.second});
            G[tmp.first + 1][tmp.second] = 0;
        }
        if (tmp.second - 1 >= 0 && G[tmp.first][tmp.second - 1] == 1)
        {
            q.push({tmp.first, tmp.second - 1});
            G[tmp.first][tmp.second - 1] = 0;
        }
        if (tmp.second + 1 < 8 && G[tmp.first][tmp.second + 1] == 1)
        {
            q.push({tmp.first, tmp.second + 1});
            G[tmp.first][tmp.second + 1] = 0;
        }
    }
}
```

接續前面的 BFS 函數，當我搜索到陸地時為了避免重複計算到相鄰的陸地，所以先用 BFS 預先將路地都變成水域。

輸入及結果

```
Enter the filename: input_hw7.txt
Number of Islands(Pattern1): 4
Number of Islands(Pattern2): 7
Number of Islands(Pattern3): 4
Number of Islands(Pattern4): 2
Number of Islands(Pattern5): 3
Number of Islands(Pattern6): 3
Number of Islands(Pattern7): 3
Number of Islands(Pattern8): 1
Number of Islands(Pattern9): 2
Number of Islands(Pattern10): 2
Number of Islands(Pattern11): 0
```

輸入要求就只有一個，就是在第一步依照指示輸入要測的檔案，隨後程式就會依照 8X8 矩陣讀入資料，並計算陸地數量後回傳。

注意!!之所以會有 Pattern11 是因為我的寫法是 `in.eof()`，在讀取完成前會再讀取一次，然後我 G 矩陣預設為元素全為零的矩陣，所以就會產生結果為零的 pattern

心得

這次作業真的蠻有趣的，沒想到 BFS 居然可以這樣使用，稍微變動一下就要另外去思考很多問題，再平板上寫寫畫畫一段時間想出這解法。